

11

Vue.js Fundamentals in a Todo App

This chapter is entirely devoted to Vue.js, the Node.js npm, and the Vue CLI. These tools help developers to scaffold Vue.js projects with different configurations based on the user's options. This chapter also describes the Vue component's features and what you can do with them. Not only that, but you will also learn about the real-world structuring of a frontend web framework. We will do everything that I just mentioned in a Todo app using TypeScript.

In this chapter, we will cover the following topics:

- Starting a project using the Vue CLI
- Files and folders generated by the Vue CLI
- Getting started with a Vue component
- Common features in a Vue component

Technical requirements

Here is what you need to complete this chapter:

- **Visual Studio Code:** <https://code.visualstudio.com/>
- **npm:** The Node Package Manager from <https://nodejs.org/en/>
- **Vue CLI:** <https://cli.vuejs.org/>

And the finished repository for this chapter can be found at the following link: <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter11/vue3-typescript-todo>

Starting a project using the Vue CLI

Vue CLI is the standard development tool for starting a Vue.js project. The CLI lets you add different supports in a project, such as support for Babel, ESLint, TypeScript, **Progressive Web Apps (PWAs)**, PostCSS, unit testing, and end-to-end testing.

Make sure you have the npm that we installed in *Chapter 2, Setting Up a Development Environment*. If you have forgotten to install the npm, you can go to <https://nodejs.org/en/> and install the latest **Long Term Support (LTS)** version of Node.js.

If you weren't able to install the Vue.js CLI in *Chapter 2, Setting Up a Development Environment*, you can do so now by running the following command:

```
npm install -g @vue/cli
```

The preceding command installs the Vue CLI globally. The last part of the npm command is the package name, while `-g` means *globally*.

After installing the Vue CLI, let's create our first Vue.js app and build a simple Todo app to try out the common features of the Vue component:

1. Run the following command to create the directory of your Vue.js app:

```
vue create todo-app
```

The `vue create` command will also trigger the Vue CLI to open up the series of configurations for the Vue app.

2. You should see the first list of options requesting the preset of the app, as per the following screenshot:



```

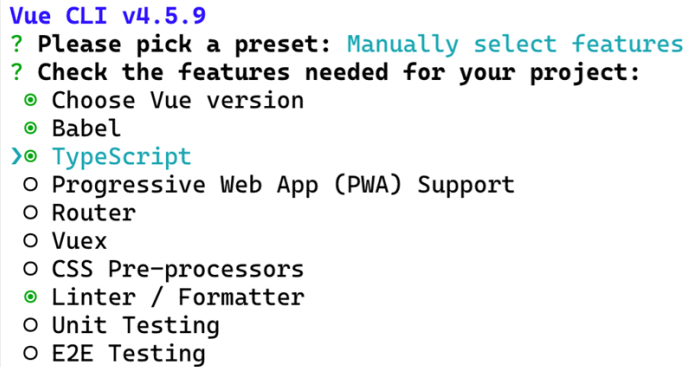
Vue CLI v4.5.9
? Please pick a preset:
  Default ([Vue 2] babel, eslint)
  Default (Vue 3 Preview) ([Vue 3] babel, eslint)
> Manually select features

```

Figure 11.1 – Vue CLI configuration options

Choose `Manually select features` and press *Enter* to see the various options to configure.

3. The Choose Vue version, Babel, and Linter / Formatter options are enabled by the Vue CLI. Let's also turn on TypeScript by pointing the cursor to TypeScript, as shown, and then press the *spacebar*:



```

Vue CLI v4.5.9
? Please pick a preset: Manually select features
? Check the features needed for your project:
  ☐ Choose Vue version
  ☐ Babel
  > ☒ TypeScript
    ☐ Progressive Web App (PWA) Support
    ☐ Router
    ☐ Vuex
    ☐ CSS Pre-processors
  ☒ Linter / Formatter
    ☐ Unit Testing
    ☐ E2E Testing

```

Figure 11.2 – Adding TypeScript to the Vue.js CLI configuration

Press *Enter* to add TypeScript support to the Vue.js project we are creating.

4. Let's use version 3 of Vue.js in the project by selecting the `3.x (Preview)` option, as shown in the following screenshot:



```

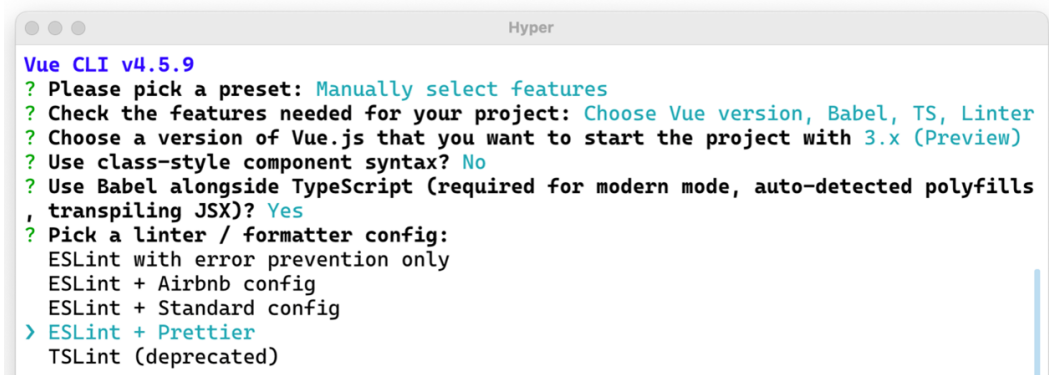
Vue CLI v4.5.9
? Please pick a preset: Manually select features
? Check the features needed for your project: Choose Vue version, Babel, TS, Linter
? Choose a version of Vue.js that you want to start the project with
  2.x
  > 3.x (Preview)

```

Figure 11.3 – Choosing Vue.js 3

Press *Enter* to choose Vue.js 3.

5. The next question is whether we would like to use `class-style` component syntax. Press *Enter* to refuse. The default selected value in this question is `No`, so pressing *Enter* answers no to this configuration. We will not be using `class-style` component syntax.
6. Another question is whether we would like to use `Babel` alongside `TypeScript`. Press *Enter* to accept. `Babel` is a transpiler that will help your `Vue.js` app autodetect polyfills, making a `Vue.js` app compatible with most browsers.
7. As regards `linter` / `formatter` while we are writing our code, let's choose `ESLint`, a static code analysis tool, plus `Prettier`, an opinionated code formatter, as shown in the following screenshot:



```

Vue CLI v4.5.9
? Please pick a preset: Manually select features
? Check the features needed for your project: Choose Vue version, Babel, TS, Linter
? Choose a version of Vue.js that you want to start the project with 3.x (Preview)
? Use class-style component syntax? No
? Use Babel alongside TypeScript (required for modern mode, auto-detected polyfills
, transpiling JSX)? Yes
? Pick a linter / formatter config:
  ESLint with error prevention only
  ESLint + Airbnb config
  ESLint + Standard config
> ESLint + Prettier
  TSLint (deprecated)

```

Figure 11.4 – Formatter configuration in the Vue CLI

Press *Enter* after using your arrow keys to point to the `ESLint + Prettier` configuration.

8. For the next option, which is for an additional lint feature, select `Lint on save`.
9. Then, select dedicated configuration files for the preferred placing of configuration for `Babel`, `ESLint`, and so on.
10. Lastly, don't accept saving the `preset` for future projects for now.

The `Vue CLI` will start creating our project, initialize the project with `Git`, install the `CLI` plugins, and add `JavaScript` packages.

Now, start your `VS Code` text editor and install the extensions, namely, **Prettier – Code formatter**, **Vetur**, and **vscode-icons**, as you can see in the following screenshot:

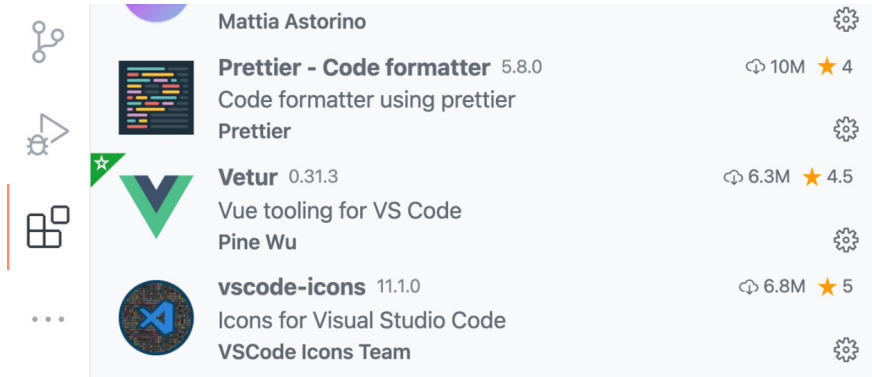


Figure 11.5 – VS Code extensions

Prettier reformats code to make it more readable, Vetur is the Vue language server, while vscode-icons is designed to bring icons to your VS Code text editor's files and folders.

You can now use your Visual Studio Code to open up the project and see the files and folders that the Vue CLI has generated for you:

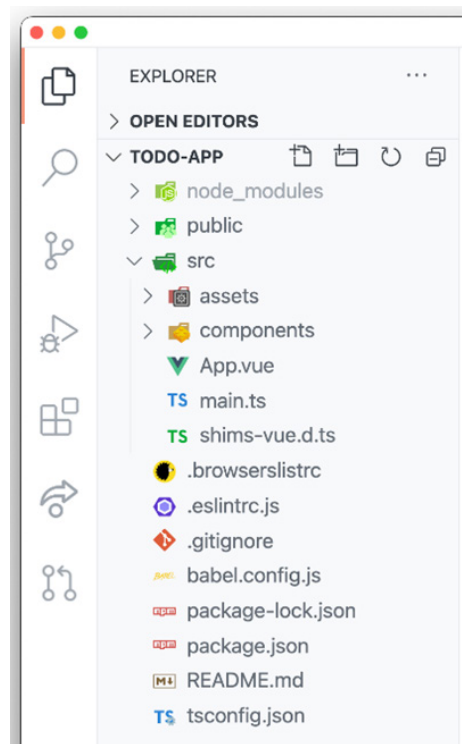


Figure 11.6 – Files and folders generated by the Vue CLI

The preceding screenshot shows the generated files and folders of the Vue CLI, and we're going to talk about these in the next section.

Files and folders generated by the Vue CLI

The Vue CLI has generated the files and folders needed to get started with our Vue.js development. Let's look at each of these in turn:

- `node_modules` folder: This folder contains the libraries downloaded from the npm.
- `public` folder: This folder contains the HTML file and favicon. You will only see one HTML file in the public folder, hence the term single-page application.
- `src` folder: This folder is the directory where we will write business logic, create Vue file components, and JavaScript or TypeScript files.
- `.browserlistrc`: This file is a tool for describing the app's target browsers.
- `.eslintrc.js`: This file is a configuration tool for ESLint.
- `.gitignore`: This file is used for not committing directories or files in Git. An excellent example of a directory that must be ignored is `node_modules`. The file size of `node_modules` is large but can always be retrieved in the project by running `npm install` in the root project directory.
- `babel.config.js`: This file is for a project-wide configuration of Babel, a JavaScript transpiler. Babel transpiles older JavaScript versions (ES6+) to lower JavaScript versions (ES5 and below) that older browsers can understand.
- `package-lock.json`: This file locks the dependencies of the project to specific versions of packages.
- `package.json`: This file holds the information of our project's dependencies and scripts, which we can use to run in the terminal.

There are three elements of `package.json` that I would like you to remember:

`scripts`: The `scripts` property is where you can declare some custom CLI commands. The Vue.js CLI has three default npm scripts in our project, namely, the `server` for running the Vue project through the `vue-cli-service serve` command, the `build` for building the Vue project for deployment, and the `lint` for running the linter of the Vue project. The idea here is that we can convert long commands into short commands by merely creating a key and value pair in the script's block.

dependencies: The `dependencies` property is simply a list of critical packages that our app needs in order to run. Any libraries that we install through the `npm install` command will be listed inside the `dependencies`.

devDependencies: The `devDependencies` property is a list of packages that help developers write apps. We can explicitly tell the `npm` that a library must be included in `devDependencies` by adding the `-D` flag. For instance, the `npm install -D prettier` command will add the `prettier` package to `devDependencies`. Also, `devDependencies` don't get compiled or included when you run the `npm build` command for the app.

- `tsconfig`: This file lets us declare the compiler options required whenever we compile the project. It also indicates that the project is a TypeScript project.

Now we have a basic understanding of the generated files and folders that the Vue CLI has scaffolded for us. Let's run the application to test whether it will appear on the browser by running the following command in your VS Code's terminal:

```
npm run serve
```

Head to `localhost:8080` in your browser to see the Vue.js app running, as shown in the following screenshot:

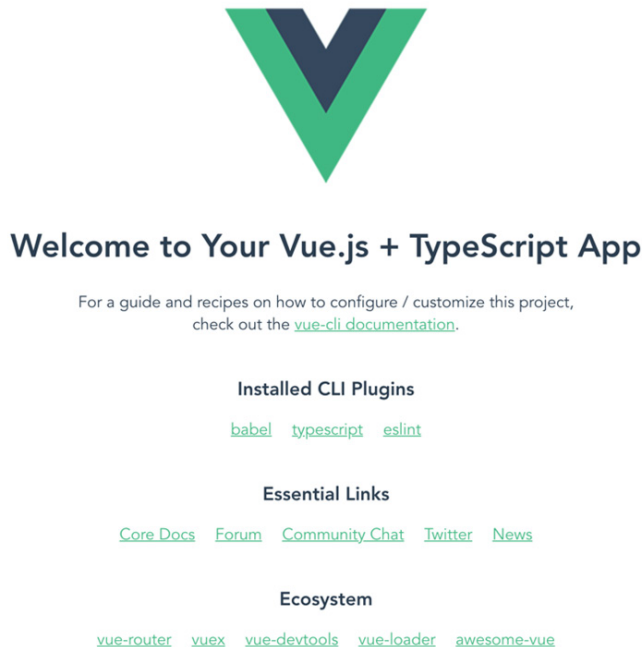


Figure 11.7 – Starting UI

You will see on the browser a welcome message with the Vue.js logo after starting the Vue.js application. The default UI has some external links concerning documentation of the features in Vue.js.

After seeing our Vue.js app running in the browser, we can now talk about the Vue component and its parts in the next section.

Getting started with a Vue component

Vue components are the main building blocks for Vue.js applications. A component consists of `template` syntax, `script`, and `style`.

Let's check out the generated code in the root component, `App.vue`, of our application:

```
<template>
  
  <HelloWorld msg="Welcome to Your Vue.js + TypeScript App" />
</template>

<script lang="ts">
import { defineComponent } from "vue";
import HelloWorld from "../components/HelloWorld.vue";

export default defineComponent({
  name: "App",
  components: {
    HelloWorld
  }
});
</script>

<style>
#app {
  font-family: Avenir, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: #2c3e50;
  margin-top: 60px;
}
```

```
}  
</style>
```

Here are some brief and straight-to-the-point explanations of the parts of the `Vue.js` component:

The `template` syntax section is where we bind the state of our application to the HTML DOM of our choice. You can also see that the `HelloWorld` component is being used in the template. We will also be going to create some Vue components in the latter part of this chapter.

The `script` section is where we can write JavaScript/TypeScript code to develop some business logic in our app, import files or packages, and define our application's states. A **state** is an object where we can store property values and render them on the UI.

You can see in the script block that `HelloWorld` was imported and registered in the `components` property to allow the `App.vue` component to render `HelloWorld` in the `template` section.

The `style` section is where we define the styling of the component we are making. You can use CSS preprocessors here, such as SASS, LESS, Stylus, and PostCSS, but these require loader plugins in order to work.

Now, remove the `HelloWorld` component from the `template` and `script` sections, and then delete it from the `components` directory. You should only see the `Vue.js` logo on the browser after removing the `HelloWorld` component from the application.

Let's write our first `Vue.js` component in the next section.

Writing a Vue component

Let's create a Vue component (the file ends in `.vue` format) inside the `components` directory and name it `TodoForm`. The filename should look like this – `TodoForm.vue`. Then write the following code:

```
<template>  
  <h1>TodoForm Works!</h1>  
</template>
```

We are not required to include any JavaScript/TypeScript code and CSS styling if we are only going to render a message in the UI. Now import it into the `App.vue` file like so:

```
<template>  
  
```

```
<TodoForm />
</template>
<script lang="ts">
import { defineComponent } from "vue";
import TodoForm from "../components/TodoForm.vue";

export default defineComponent({
  name: "App",
  components: {
    TodoForm,
  },
});
</script>
```

TodoForm is imported in the same way that HelloWorld was imported and rendered in the UI. Now hit the **Save** button and see the results in the browser:



TodoForm Works!

Figure 11.8 – Vue.js component

The implementation we did to create a Vue component was a quick-and-dirty way of making it, but we will add some features to it in the next section.

Common features in a Vue component

In this section, we will use some of the common features in a Vue component while building our Todo app. There are many directives (custom HTML attributes), events, and interfaces that we can use in Vue.js, but most of them are rarely used, which you'll tend to forget over time. So, in this quick demo of the Vue.js Todo app, we will only use the most common events, directives, and APIs of a Vue component.

So let's get started.

Writing local states in a Vue component

Since we are using TypeScript in this project, the first thing that I usually do is write the models that I will need in the app. So let's create a folder inside the `src` directory and name it `models`. Then, create a TypeScript file inside the `models` folder (`src | models | todoModel.ts`) and add the following code:

```
export type TodoType = {  
  id: string;  
  done: boolean;  
  content: string;  
};  
// OR  
export interface TodoModel {  
  id: string;  
  done: boolean;  
  content: string;  
}
```

The shape or model of a state can be written using the type or interface. They are both the same; the difference is that you can't implement two or more interfaces, whereas the type allows us to do that. We will use a simple model here that has `id`, `done`, and `content` primitive type properties.

Now we are going to require some third-party packages. These are `uuid` for generating a Universally Unique Identifier for our app, and `@types/uuid` for the Type definitions of the UUID library. Our Vue.js TypeScript project will give us IntelliSense when using the `uuid` library, and lastly, Bootstrap 5 for styling our components.

So let's run the following command to start installing the packages:

```
npm i uuid @types/uuid bootstrap@next
```

The `i` is short for install, and the `@next` suffix of Bootstrap is explicitly indicating that we want the alpha or beta version of the library because the stable version of Bootstrap is v4 at the time of writing. However, you don't need `@next` after `bootstrap` if you see that the stable version of Bootstrap on the npm website is version 5.

Now let's use the Bootstrap package in our Vue.js project by replacing the styles generated in `App.vue` with the following code, which imports Bootstrap's CSS:

```
<style>
@import "../node_modules/bootstrap/dist/css/bootstrap.css";
</style>
```

After importing the Bootstrap in the root component, let's use the container from the Bootstrap package, one of the most basic layout element in Bootstrap. We can use it in a `div` tag and wrap the Vue logo and the `ToDoForm` component like so:

```
<template>
  <div class="container">
    
    <ToDoForm />
  </div>
</template>
```

Hit **Save** to save the updated file and look in your browser to see the changes in respect to the layout. Then, let's also update `ToDoForm.vue` inside the components folder by changing the HTML and adding a state to the component.

Let's edit the message in the `ToDoForm` component with the following code:

```
<template>
  <div class="mb-4">
    <h1>Vue 3</h1>
    <h2>TypeScript Demo {{ version }}</h2>
  </div>
</template>
```

The new message in `ToDoForm.vue` is now `Vue 3 TypeScript Demo`, but the mustache syntax for text interpolation, `{{ version }}`, with the word `version` inside, will not be showing in the UI because this is a form of data binding between the UI and a state.

So let's now create a state for the `ToDoForm` component:

```
<script lang="ts">
import { defineComponent, ref } from "vue";

export default defineComponent({
```

```

name: "TodoForm",

setup() {
  // local state
  const version = ref("v1");

  return { version }; // same as { version:version }
},
});
</script>

```

We need `defineComponent` and `ref` from the Vue library. The `defineComponent` component is self-explanatory. `ref` is for creating and tracking a local state, the state for any changes that render it reactive, which rerenders the UI whenever the value of the state changes. We call `ref` and pass an initial value `v1` of the string type and store it in the `version` variable. Then we need to return it as a property of an anonymous JavaScript object.

Save and see the Vue app on the browser. You should see the **TypeScript Demo** message followed by **v1**, as shown here:



Figure 11.9 – Text interpolation data binding using a mustache

That's how easy it is to do the data binding. Now let's create another two local states, one with an explicit type and one with an implicit type.

Let's first import the `TodoType` model that we created:

```
import { TodoType } from "@/models/todoModel";
```

The @ sign in the path is a resolve alias provided by `tsconfig` to help us easily reach exported codes or libraries directly without writing too many dots and slashes. You can check out the `tsconfig` file's properties to see the value of the `baseUrl` and the paths:

```
// implicit type safe
const version = ref("v1");
const newTodo = ref(""); // ref<string> is too verbose

// explicit type safe
const todos = ref<TodoType[]>([]);
```

Update the `setup` function of your component with the new states indicated above. The `version` and `newTodo` states are written with implicit types. TypeScript knows what type you are passing, and it can infer what type you are returning from `ref`. You can hover your mouse over `version` and `newTodo`, and you'll see that they are string types.

The explicit way of telling the TypeScript compiler the initial value type is by using a generic in `ref`. You can see in the preceding code that `todos` is a `TodoType` type of array. This is a good way of writing a state's static typing for readability purposes, and not only for IntelliSense.

You'll notice that we didn't explicitly add a type for the `version` and `newTodo`, both string types, because it is easy to tell what type a value is if it's a primitive type, such as a Boolean, string, and number.

OK, so we are done writing states in our `TodoForm` component and rendering it. Let's now add functionality to our `TodoForm` component.

Adding a function in a Vue component

Writing a function in a Vue component is dead simple. The first step involves importing `uuid` below `TodoType` to generate a UUID for the ID of our model:

```
import { v4 as uuidv4 } from "uuid";
```

We are importing `v4` and renaming it to `uuidv4` for readability.

Now we can start adding a function where we can create a new `Todo` object:

```
// explicit type safe
const todos = ref<TodoType[]>([]);

function addNewTodo(): void {
```

```
    if (!newTodo.value) return;

    todos.value.push({
      id: uuidv4(),
      done: false,
      content: newTodo.value,
    });

    console.log("newTodo:", newTodo.value);

    newTodo.value = "";
  }
}
```

The `addNewTodo` function, which returns `void`, checks whether `newTodo.value`, which we will connect to the input field, is empty. This is how we can get the value of our state, by accessing the `value` property. Then, if the value is not empty, we will push a `Todo` object to the `todos`. The `Todo` object will have a generated `id`, `done` set to `false`, and a `content` property whose value will come from `newTodo`. The function also logs the string value of `newTodo` and then sets it to an empty string to remove the user's input.

Then, let's include the `addNewTodo` function in the returning object of the `setup`, along with the `todos` and `newTodo`, like so:

```
return { version, todos, newTodo, addNewTodo };
```

As you can see in the preceding code, returning the states and function will make them available to be used in the template section of the Vue component.

Now place this HTML form below the `div` tag, which is wrapping the Vue3 TypeScript demo:

```
<form @submit.prevent="addNewTodo">
  <div class="mb-5">
    <label for="newTodo" class="form-label">New Todo
    </label>
    <input
      class="form-control"
      id="newTodo"
      placeholder="what's on your mind?"
      v-model="newTodo"
```

```
      name="newTodo"
    />
    <div class="m-2">
      <button type="submit" class="btn btn-primary">Add
        New Todo</button>
    </div>
  </div>
</form>
```

`@submit`, which is a submit listener, invokes the `addNewTodo` function we created every time a button with the `type="submit"` attribute gets hit. The `newTodo` input is binded to the input's `v-model`, directive which is a directive to create a two-way data binding between the input and the model or state. The rest of the details are basic HTML syntaxes and Bootstrap styling.

Save the file and check the browser. You should see the simple adding of the todo form under **TypeScript Demo v1**:



The screenshot shows a web application interface. At the top, there is a large green and dark blue 'V' logo representing Vue.js. Below the logo, the text 'Vue 3' is displayed in a large, bold, black font, followed by 'TypeScript Demo v1' in a slightly smaller, bold, black font. Underneath this, the text 'New Todo' is displayed in a smaller, regular black font. Below 'New Todo' is a text input field with the placeholder text 'what's on your mind?'. Below the input field is a blue button with the text 'Add New Todo' in white.

Figure 11.10 – Simple form

The form is not yet ready to render what we add in the input, but we can see in the console whether it's working. Now open up your Chrome DevTools and go to the **Console** tab. Write check all emails in the input field and click the **Add New Todo** button of the app to see that it is being registered in the console:

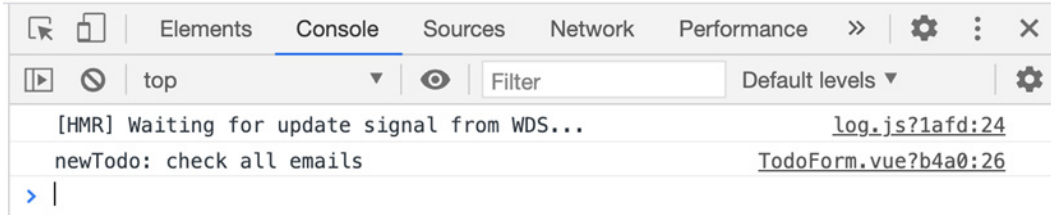


Figure 11.11 – Console log

The preceding *Figure 11.11* shows the **Console** tab and the **newTodo: check all emails** log in the DevTools. This is a quick **Proof of Concept (PoC)** that we can use to create functions in our Vue component.

We just did a PoC for a function in our Vue component. Let's now render the list of todos in our component.

Looping in an array in a Vue component

Now we can add the UI for the array of `TodoType`, which is a collection, meaning we write a loop of the todos and render each item in the collection or list. We can do that by using the `v-for` directive.

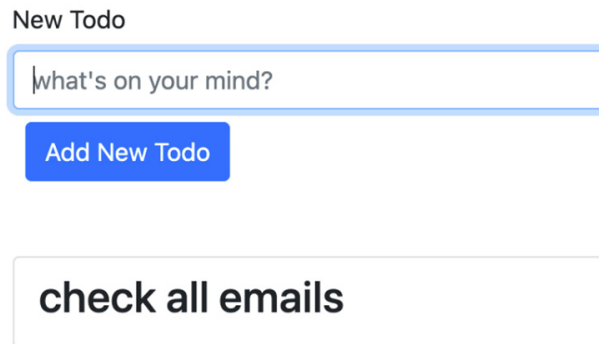
Write the following below the form tag:

```
<div>
  <ul class="list-group">
    <li class="list-group-item" v-for="todo in todos"
      :key="todo.id">
      <h3>
        {{ todo.content }}
      </h3>
    </li>
  </ul>
</div>
```

The code does a `for each` loop in the `todos` and renders every `todo`'s `content` property in the list. You will notice another Vue.js directive, `:key="todo.id"`, which aims to help the Vue.js runtime determine which specific object of the list alone needs to re-render whenever the `todos` state changes. In return, the rerendering of the DOM is more performant without the key.

The colon prefixed to the key directive is shorthand for the `v-bind` directive, which is used for one or more data binding attributes. The key directive (`key=" "`) needs a string value or a number value to work. A good example of what to pass in a key directive is an ID. However, the key directive does not have to be an ID, what matters is that it has to be unique.

Try the form again by writing `check all emails` in the field and hitting *Enter*. You should see **check all emails** in the UI after adding it, as shown in the following screenshot:



The screenshot shows a web interface for a todo app. At the top, there's a section titled 'New Todo'. It contains a text input field with the placeholder text 'what's on your mind?' and a blue button labeled 'Add New Todo'. Below this, there's a list of todos. The first todo in the list is 'check all emails', which is displayed in a bold, black font.

Figure 11.12 – Rendering a list of todos

Now we can render our todos in the console logs and the screen of the browser.

What if we want to render something if the `todos` state is empty? What if we want to mark the todo as done and change the style of the todo from plain to with strikethrough? That's what we are going to write next.

If-else conditions in a Vue component

We have two goals here in this section. Our first goal is to show a message of an empty list with a sad emoji face if the `todos` state is empty. You can get the emoji from the following link: <https://emojipedia.org/>.

If the `todos` state is not empty, we can start showing the list of todos and hide the `Empty list` with a sad face message.

Place the code for an empty list UI between the form tag and list of the todos, like so:

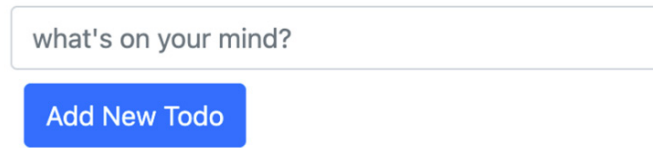
```
</form>
<div v-if="todos.length === 0">
  <h3>Empty list 😞</h3>
</div>
```

```
<div v-else>
  <ul class="list-group">
```

In this code, we use the `v-if` directive conditionally to render a block and the optional `v-else` directive as `else` in the condition.

Check out the browser and refresh your page to see the new UI, *Figure 11.13*. You should see the sad emoji face while the `todos` state is empty:

New Todo



Empty list 🙄

Figure 11.13 – Empty list with the sad face

The empty list with the sad face immediately goes away when the `todos` state is no longer empty. Easy right?

Our second goal in this section is to create a functionality where we can update the `done` property of the `Todo` object. We will let the user know if `done` has turned into `true` by marking the `todo` content with a strikethrough. The strikethrough is a simple CSS style that we are also going to write in a bit.

So let's start. Write a new function named `toggleDone` that takes a `TodoType` object like so:

```
function toggleDone(todo: TodoType): void {
  todo.done = !todo.done;
}
```

The `toggleDone` function updates the `done` property to its opposite value every time the function gets invoked.

Now let's create a CSS class by putting it inside a `style` section like so:

```
<style>
.mark {
```

```
text-decoration: line-through;
}
</style>
```

The style in the code is a line-through that will give the todos content a strike-through on it that will serve as a mark.

Now update the `h3` tag with these new attributes:

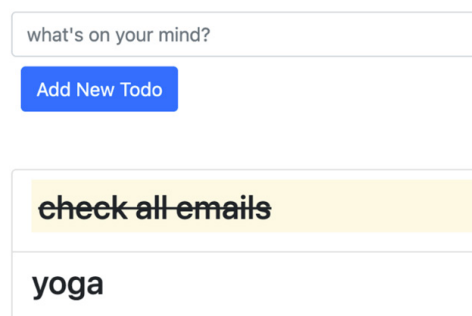
```
<h3
  :class="{ mark: todo.done }"
  style="cursor: pointer"
  @click="toggleDone(todo)"
>
```

`:class`, which is shorthand for `v-bind:class`, is a class directive that dynamically toggles the class depending on the Boolean value of `todo.done`. The `mark class` style gets activated if `todo.done` is `true`, and gets deactivated if `todo.done` is `false`. Simple yet helpful.

You will also notice here that there is `@click`. The `@` sign is shorthand for the `v-on` directive, which listens to DOM events to run or call JavaScript functions. You can see all the events in the IntelliSense of your editor when you start typing the `@` sign in any HTML tag. You will get `mouseover`, `blur`, `keyup`, and so on.

Let's check out the `toggleDone` function. The `toggleDone` function has also been placed, and we are passing the `todo` object that comes from the `v-for` looping directive you will find in the `<li>` element. Now we can test the new functionality of the app.

Then, enter `check all emails` and `yoga`. Then, click on **Check all emails** to mark it with strike-through, as shown in the following screenshot:



what's on your mind?

Add New Todo

~~check all emails~~

yoga

Figure 11.14 – Mark what's done with a strike-through

The strike-through means that the `done` property in the `todo` object was set to `true`.

Let's create more functionalities, such as marking all items in the list, deleting all items in the list, or just removing one item.

Write a `removeTodo` function that takes a number type like so:

```
function removeTodo(index: number): void {
  todos.value.splice(index, 1);
}
```

We are going to remove a specific `todo` object from the `todos` list.

Include `removeTodo` in the return object of the `setup` function of the component:

```
return {
  ...,
  removeTodo,
};
```

`removeTodo` is now ready to use in the HTML.

Now let's update the template syntax, particularly the `li` tag:

```
<ul class="list-group">
  <li
    class="list-group-item d-flex flex-row justify-content-
      between align-items-center"
    v-for="(todo, index) in todos"
    :key="todo.id">
    <h3
```

We are updating the `class` and `v-for` of the `li` tag. You will notice that there's a second parameter, `index`, in `v-for`. That's the index of the `todo` item that we get from looping. And yes, we have access to that.

Next, we add the following code to the `h3` tag:

```
<button
  type="button"
  class="btn btn-warning"
  @click="removeTodo(index)"
>
```

```
    ✓ Done & Remove
  </button>
```

We pass the index to the `removeTodo` function and name the button `Done & Remove` with a check emoji.

Now let's write two more functions and name them `markAllDone` and `removeAllTodos`, like so:

```
function markAllDone(): void {
  todos.value.forEach((todo) => (todo.done = true));
}

function removeAllTodos(): void {
  todos.value = [];
}
```

`markAllDone` sets all todos done to `true`, while `removeAllTodos` sets the todos to an empty array, removing all items in the list.

Then, include `markAllDone` and `removeAllTodos` in the return statement like so:

```
return {
  ...,
  markAllDone,
  removeAllTodos,
};
```

`markAllDone` and `removeAllTodos` are now available to use within the `TodoForm` component's template syntax.

Now, let's add two new buttons below the **Add New Todo** button:

```
<div class="m-2">
  <button
    type="button" class="btn btn-danger"
    @click="removeAllTodos">
    Remove All
  </button>
</div>
<div class="m-2">
```

```
<button  
  type="button"  
  class="btn btn-success"  
  @click="markAllDone">  
  Mark All Done  
</button>  
</div>
```

The two new buttons handle the `removeAllTodos` and `markAllDone` functions. Run the app and check the browser. We will get the following screenshot:

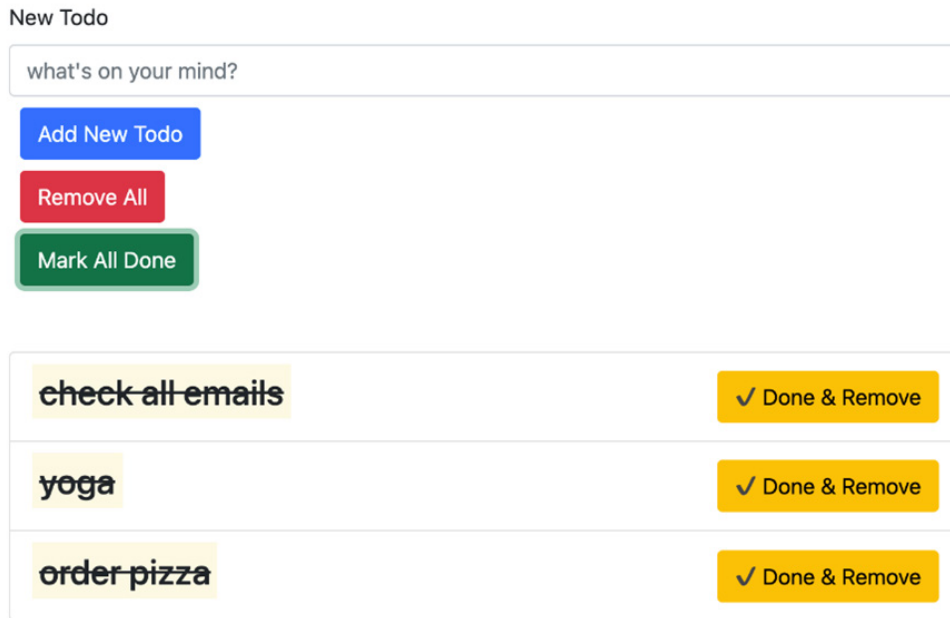


Figure 11.15 – Remove All and Mark All Done

Add another todo title, `order pizza`, and click the **Mark All Done** button. You should see that all todos were struck through, as shown in *Figure 11.15*. Then, hit the **Remove All** button to remove all todos.

I am sure that you find Vue.js easy to use, and it's not that hard to add functionalities. Now, let's make it a little more complicated by passing down states from the parent to the child component in the next section.

Creating and passing props

There are instances when a parent component has to pass a state or a function to its child component to render UI or create functionality, and we do it through props. Props is a term that is also used in other JavaScript frameworks or libraries, such as Angular, React, Svelte, and Ember.

To start, let's define props in the `TodoForm` component. Import `PropType` from `Vue` to get `typesafe`. You won't need this if your Vue project uses JavaScript, but it will be beneficial if you are using TypeScript:

```
import { defineComponent, ref, PropType } from "vue";
```

After importing `PropType`, create a model above `defineComponent` and name it `Props`, like so:

```
import { v4 as uuidv4 } from "uuid";  
type Props = {  
  title: string;  
  subTitle: string;  
};  
export default defineComponent({
```

You can use a type or interface to create a model. Now, let's define `about` props of `TodoForm` and place it before the `setup` like so:

```
  name: "TodoForm",  
  props: {  
    about: {  
      type: Object as PropType<Props>,  
      required: true,  
    },  
  },  
  setup() {
```

We are naming our props as `about`. `about` is a required prop, and it is an object of `PropType` with the shape of `Props`.

The next thing is to create a state name with a `title` string and `subTitle` string properties, like so:

```
// Vue 3
setup() {
  const about = ref({
    title: "Vue 3",
    subTitle: "TypeScript demo",
  });

  return {
    about,
  };
},
// Vue 2
/*
data:() => ({
  about: {
    title: "Vue 3",
    subTitle: "TypeScript demo"
  }
})
*/
```

Import `ref` and use it to create a state, store it in `about`, and then return `about`. You will notice here that I've put a `Vue 3` label on top of the `setup` and a `Vue 2` label on top of the `data` function. These are two different ways of writing a component in `Vue.js` that you will encounter when you start looking for a sample `Vue` project on the internet. I would say that 90% of what you will see at the time of this writing until the next few months will be written in `Vue.js 2` because `Vue.js 3` was just released in September 2020 and the majority of `Vue.js` communities and library authors haven't yet adopted the latest `Vue.js` version.

Before we can pass `about` of the `App.vue` component, we need to update the setup of `TodoForm`. So go back to the `TodoForm.vue` component and update the setup like so:

```
setup(props) {
```

There are optional parameters in the setup that we can use. The first parameter is `props`, which is the entry point of any state and functions that we will pass into a child component.

Let's map `props` to the `template` syntax of `TodoForm`, replacing the Vue 3 TypeScript demo message:

```
<template>
  <div class="mb-4">
    <h1>{{ about.title }}</h1>
    <h2>{{ about.subTitle }} {{ version }}</h2>
  </div>
```

The idea is that the `TodoForm` welcome message will come from its parent component from now on.

Now, let's update the `template` section of the `App.vue` component:

```
<TodoForm :about="about" />
```

Here we are passing the `about` state of `App.vue` to `about`, `:about`, and `props` of `TodoForm`.

You should see that the UI is still working, as shown in the following screenshot:

Vue 3 TypeScript demo v1

New Todo

Add New Todo

Remove All

Mark All Done

Empty list 🙄

Figure 11.16 – `TodoForm` with `about` props

The Vue3 TypeScript demo is rendering again. What if you don't see the Vue 3 TypeScript demo and you want to debug it? We can use a console log in the props, which we will do in the next section.

Life cycle hooks in a Vue component

Life cycle hooks are functions or methods that get called automatically at each specific point in the life cycle of the component. We can tap into key events in the component's life cycle to write business logic in our app.

Let's use the most frequently used life cycle hooks, which are `onMounted` in Vue 3 or `mounted` in Vue 2. `onMounted` runs or triggers when the DOM is mounted:

```
import { defineComponent, ref, PropType, onMounted } from
"vue";
```

We are importing `onMounted` from Vue.

Then we use `onMounted` and place it before the `return` statement of the `setup` function like so:

```
onMounted(() => console.log(props.about.title));
return {
```

We are logging here the value of the `title` property of `about`, which you will see when you open your Chrome DevTools or Firefox DevTools, as shown in the following screenshot:

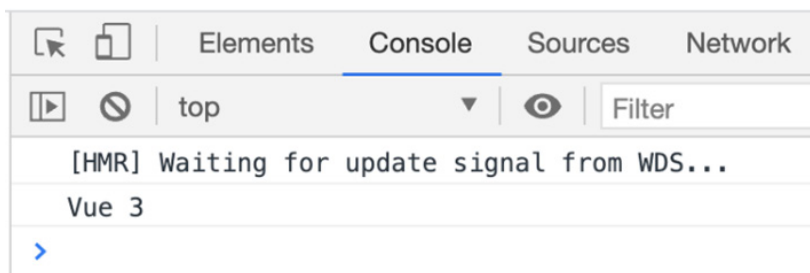


Figure 11.17 – Console logging props

You can see the Vue 3 log from the `about` props passed by `about` the state of the `App.vue` component. `onMounted` is what you will always use whenever you automatically render data from a web service on the UI for your web visitors. We will do `onMounted` with HTTP requests in our next app, the real-world Vue.js app with ASP.NET Core.

That's it. You can play around with the Todo app we just created. Now let's summarize all the things you have learned in building this simple Todo app.

Summary

There are so many things that we've done in this chapter. You learned that the Vue CLI is a great tool for scaffolding a project that saves developers a lot of time. You learned the parts of a Vue component, namely, the template block, which is the UI part of the Vue, the script section for writing business logic, and the style block for styling the component. You also learned how to create a Vue component and how to use the common interfaces in Vue, such as `v-for` for looping, the `v-if` conditions, and the `@` sign for writing events.

You were also able to learn how to write states in a Vue component using `ref` and how to do data binding using the colon prefix or by using the double curly braces.

Lastly, you learned what props are and how to pass props between two Vue components. You also learned when to use life cycle hooks in a Vue component for triggering a function.

In the next chapter, we will start to develop our real-world enterprise Vue.js app.

Further reading

I admit that I'm poor at remembering things. That's why, whenever I am learning a new technology in software development, I google for cheatsheets that help me to remember just the key aspects of a piece of technology. The following links are cheatsheets in Vue.js:

- **Vue Essentials cheat sheet:** vuemastery.com/pdf/Vue-Essentials-Cheat-Sheet.pdf
- **Vue.js cheat sheet:** devhints.io/vue